

山东大学 计算机科学与技术 学院

嵌入式系统原理与应用 课程实验报告

学号： 202300130183	姓名： 宋浩宇	班级： 23 级人工智能班
实验三：单片机与 PC 机主从串行通信接受 PC 发送数据并返回		
实验学时： 4		
实验目的： 1、学习串口初始化编程，加深对于单片机寄存器编程的理解，了解主从通信模式，掌握单片机数据收发流程； 2、熟悉嵌入式软件开发的实际操作流程，锻炼单片机软件开发能力。		
硬件环境： 1、STC15W 核心板 2、CPU：13th Gen Intel(R) Core(TM) i9-13980HX 2.20 GHz 3、内存：32.0 GB (31.6 GB 可用) 4、磁盘驱动器：NVMe WD_BLACKSN850X2000GB		
软件环境： 操作系统：window 10 嵌入式系统微控制器编程集成开发环境：Keil uVision5 STC 系列单片机下载烧录软件：stc-isp-v6.91K		
实验内容： 利用 Keil uVision5 编写程序实现：串口配置 19200bps，8 位数据位，1 位停止位；实现以下两个功能： 1. 单片机收到 hello 后，向电脑发送 world 2. PC 发送 4 位数字，单片机收到后将它们显示在数码管上。 串口电路结构图如图 3-1 所示，数码管所在位置与数码管电路原理图如图 3-2、3-3 所示，编译成对应的 hex 文件，通过 stc-isp 在 STC15W 核心板上烧录程序验证实验结果并撰写实验报告。 要求： 实验报告中附实验步骤、程序代码（关键处给出注释）和实验结果。		

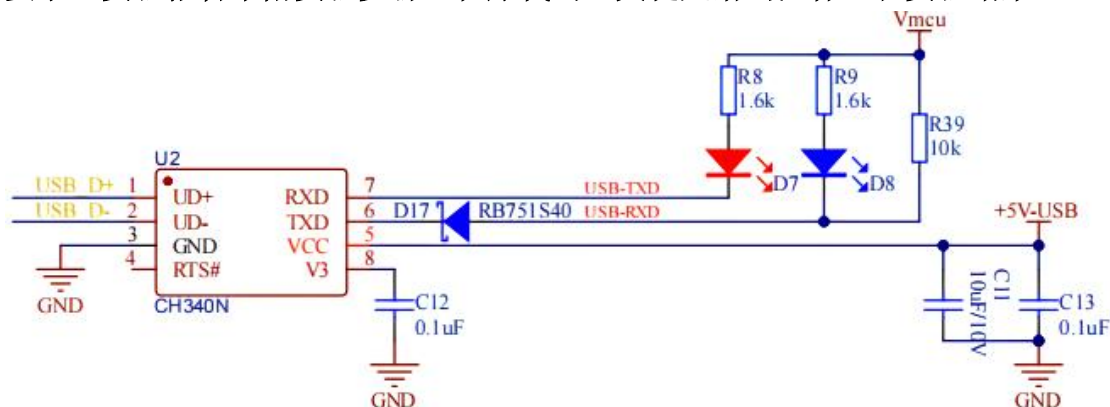


图 3-1 串口电路结构图

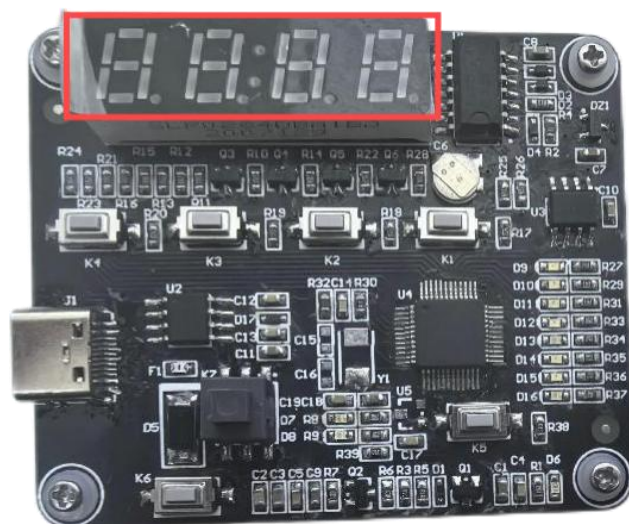


图 3-2 数码管所在位置

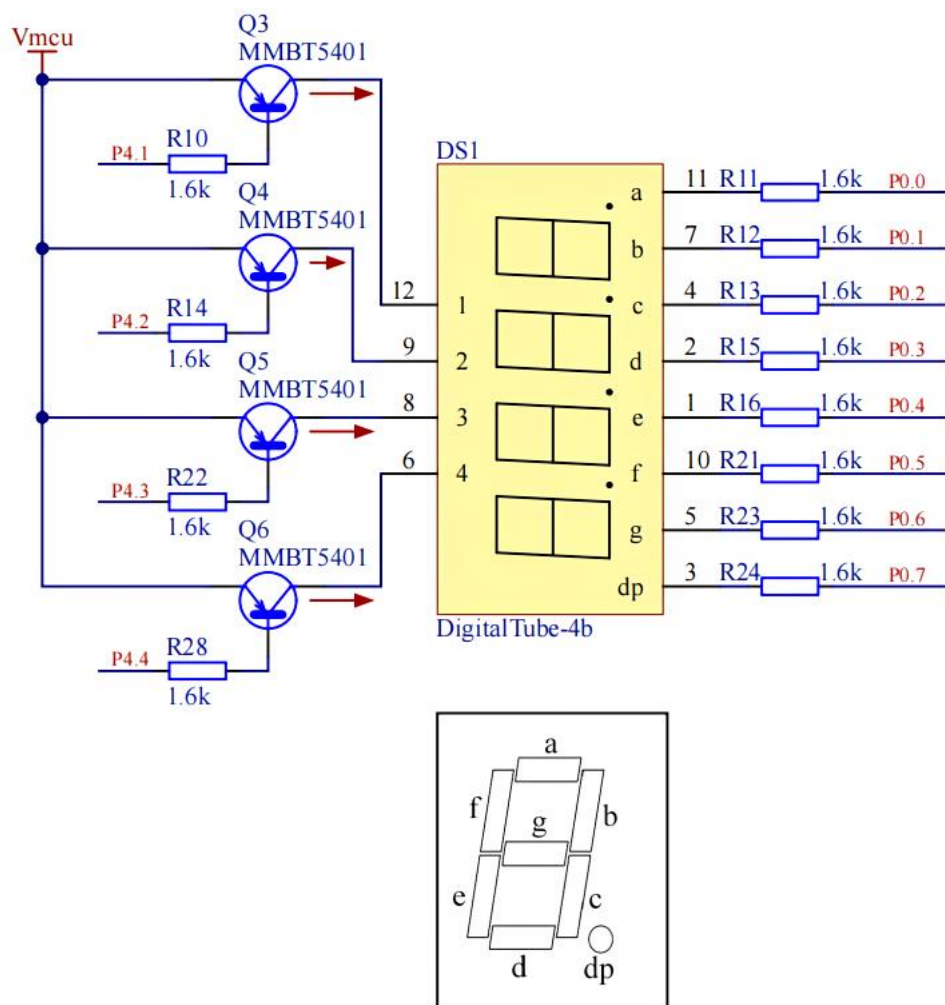


图 3-3 数码管驱动电路原理图

实验步骤、程序代码与实验结果：

实验步骤：

1. 硬件连接

使用 CH340USB 转串口模块与 STC15 单片机进行串口通信连接，将 CH340 的 TXD 接单片机

P3.0 (RXD)，CH340 的 RXD 接单片机 P3.1 (TXD)，双方 GND 共地。数码管为开发板板载电路，无需额外接线。连接完成后，将开发板通过 USB 接入电脑。

2. 工程建立与程序编译

打开 Keil5，新建工程并选择芯片型号 STC15W4K32S4，添加头文件 STC15.h 与程序源码文件。在工程设置中勾选生成 HEX 文件，即 OptionsforTarget→Output→CreateHEXFile。完成代码编写后进行编译，确保程序无错误与警告。

3. 程序烧录

打开 STC-ISP 下载软件，选择对应单片机型号，设置系统时钟为内部 IRC，频率设置为 11.0592MHz。随后载入编译生成的 .hex 文件，点击“下载/编程”，再按下开发板复位键完成烧录。观察软件提示信息，确认程序下载成功。

4. 串口调试助手配置

打开串口调试助手，选择对应串口号，并设置波特率为 19200，数据位为 8 位，停止位为 1 位，校验位选择无校验 (None)。同时勾选“发送新行”或“发送回车换行”，保证发送字符串时能够正确结束一帧数据。

5. 功能测试

在串口助手输入 hello 并发送，单片机会自动返回 world，说明串口通信与字符串识别功能正常。继续输入 4 位数字，例如 1234，发送后开发板上的四位数码管同步显示对应数字，说明串口数据解析与数码管动态显示功能正常。

6. 问题排查

若串口无法通信，需要检查 TXD 与 RXD 是否交叉连接、GND 是否共地、波特率是否一致、单片机时钟频率是否正确以及串口号是否选择正确。若发送 hello 后无 world 返回，需要检查是否勾选“发送新行”以及是否正确发送回车换行符\r\n。若数码管显示异常，则需要检查 P0、P4IO 口配置、数码管段码表、位选控制信号以及动态扫描延时设置是否正确。

代码：

```
001 #include <STC15.h>
002 #include "dataType.h"
003 #include <string.h>
004
005 typedef unsigned char u8;
006 typedef unsigned int u16;
007
008
009 u8 num_buf[4] = {0,0,0,0};
010
011
012 #define RECEIVE_BUF_SIZE 32
013
014 u8 Receive_Buf[RECEIVE_BUF_SIZE];
015 u8 Receive_Cursor = 0;
016 u8 Receive_Times = 0;
017
018 bit Frame_Ready = 0;
019
```

```

020
021 u8 code seg_code[] =
022 {
023     0xC0, //0
024     0xF9, //1
025     0xA4, //2
026     0xB0, //3
027     0x99, //4
028     0x92, //5
029     0x82, //6
030     0xF8, //7
031     0x80, //8
032     0x90, //9
033     0x88, //A
034     0x83, //B
035     0xC6, //C
036     0xA1, //D
037     0x86, //E
038     0x8E //F
039 };
040
041
042 void delay_us(u16 t)
043 {
044     while(t--);
045 }
046
047
048 void u4_num_disp(u8 index, u8 num)
049 {
050     num &= 0x0F;
051
052     switch(index)
053     {
054         case 0:
055             P0 = seg_code[num];
056             P41 = 0;
057             delay_us(200);
058             P41 = 1;
059             break;
060
061         case 1:
062             P0 = seg_code[num];
063             P42 = 0;

```

```

064         delay_us(200);
065         P42 = 1;
066         break;
067
068     case 2:
069         P0 = seg_code[num];
070         P43 = 0;
071         delay_us(200);
072         P43 = 1;
073         break;
074
075     case 3:
076         P0 = seg_code[num];
077         P44 = 0;
078         delay_us(200);
079         P44 = 1;
080         break;
081     }
082 }
083
084
085 void disp_buf()
086 {
087     u4_num_disp(0, num_buf[0]);
088     u4_num_disp(1, num_buf[1]);
089     u4_num_disp(2, num_buf[2]);
090     u4_num_disp(3, num_buf[3]);
091 }
092
093
094 void send_byte(u8 dat)
095 {
096     SBUF = dat;
097
098     while(!TI);
099
100     TI = 0;
101 }
102
103
104 void send_str(char *str)
105 {
106     while(*str)
107     {

```

```
108         send_byte(*str++);
109     }
110 }
111
112
113 void send_world()
114 {
115     send_str("world\r\n");
116 }
117
118
119 void uart_init()
120 {
121     SCON = 0x50;
122
123     /*
124      * Timer1:
125      * 模式 2
126      * 8 位自动重装
127      */
128     TMOD &= 0x0F;
129     TMOD |= 0x20;
130
131     /*
132      * STC15:
133      * Timer1 工作在 1T 模式
134      */
135     AUXR |= 0x40;
136
137     /*
138      * 11.0592MHz
139      * 19200bps
140      */
141     TH1 = 0xEE;
142     TL1 = 0xEE;
143
144     TR1 = 1;
145
146     RI = 0;
147     TI = 0;
148
149     ES = 1;
150 }
151
```

```

152
153 void uart_handler(void) interrupt 4
154 {
155     u8 rx_data;
156
157     if(RI)
158     {
159         RI = 0;
160
161         rx_data = SBUF;
162
163         Receive_Times++;
164
165         // 接收到回车/换行
166         if(rx_data == '\r' || rx_data == '\n')
167         {
168             Receive_Buf[Receive_Cursor] = '\0';
169
170             Frame_Ready = 1;
171
172             Receive_Cursor = 0;
173         }
174         else
175         {
176             if(Receive_Cursor < RECEIVE_BUF_SIZE -
177 1)
178             {
179                 Receive_Buf[Receive_Cursor++] =
180 rx_data;
181             }
182         }
183     }
184
185     if(TI)
186     {
187         TI = 0;
188     }
189 }
190
191
192 void main()
193 {
194     /* P0 推挽输出 */
195     P0M0 = 0xFF;

```

```

196     P0M1 = 0x00;
197
198     /* P4 推挽输出 */
199     P4M0 |= 0x1E;
200     P4M1 &= ~0x1E;
201
202     uart_init();
203
204     EA = 1;
205
206     while(1)
207     {
208         // 显示接收计数
209         num_buf[3] = Receive_Times & 0x0F;
210
211         disp_buf();
212
213         // 一帧接收完成
214         if(Frame_Ready)
215         {
216             // 收到hello
217             if(strcmp((char*)Receive_Buf,
218 "hello") == 0)
219             {
220                 send_world();
221             }
222
223             Frame_Ready = 0;
224
225             memset(Receive_Buf, 0,
sizeof(Receive_Buf));
        }
    }

```

实验结果：

实验成功实现了单片机与 PC 的串口通信、接收字符串并自动回复、接收数字并在数码管实时显示，同时验证了串口中断接收与数码管动态扫描显示功能，达到了实验预期目标。

